Table C.3 Built-in Django template filters *(continued)*

Filter	Description
`{{ value|unordered_list }}`	This converts a list into a series of `<li>` tags. It supports nested lists with inner lists converted into a sub `<ul>` collection. Note that the outermost output does not have `<ul>` tags wrapped around it.
`{{ value|upper }}`	This converts a string into all uppercase.
`{{ value|urlencode }}`	URL encodes a string.
`{{ value|urlize }}`	This converts text containing a URL into its clickable HTML equivalent. URLs are detected based on prefixes of *www.*, *http://*, or *https://*. Additionally, domain-only links are detected if they end in a recognized top-level domain (*.com*, *.edu*, *.gov*, *.int*, *.mil*, *.net*, or *.org*). Note that `value` may contain content besides the URL, such as a full sentence. For example `'Visit manning.com today'` is converted to `'Visit <a href="manning.com">manning.com</a> today'`.
`{{ value|urlizetrunc:"15" }}`	This is like `|urlize` but truncates the display portion of the URL if it exceeds the length given in the argument.
`{{ value|wordcount }}`	This returns the number of words in `value`.
`{{ value|wordwrap:"20" }}`	This adds newlines into a string to wrap at the given width.

`{{ value|yesno }}`
or
`{{ value|yesno:"yeah,nope" }}`
or
`{{ value|yesno:"yeah,nope,maybe" }}`
	This returns `yes` or `no` for `True` or `False`. An optional argument provides comma-separated alternate values for `yes` and `no`. A third value can be included in the argument to check `value` against `None`.

C.2.1 Loadable filters

Djangos has built-in filters that are available without using a `{% load %}` tag. Django also ships with other filters inside modules that need to be loaded to be used.

Table C.4 Loadable Django filters

Filter	Module	Description
`{{ value|apnumber }}`	humanize	This returns a number using Associated Press style, where numbers less than 10 are spelled out, while numbers greater or equal to 10 are presented as integers.
`{{ value|intcomma }}`	humanize	This converts an int or float to a string with thousands-based comma separators.
`{{ value|intword }}`	humanize	This converts large values to their written equivalent. For example, `1200000` becomes `1.2 million`.